

Student Cyber Security Management System

A menu-driven Python console application for managing student records alongside basic cyber security assessments — built as a learning project to combine simple data management with practical security awareness concepts.

Objective

The goal of this project is to design a simple, text-file-based system that allows an institution (or a student practicing system design) to:

Maintain student records (name, ID, branch, email). Evaluate each student's basic cyber hygiene (MFA usage, password strength, system updates, antivirus status). Generate an overall security score and categorize it into a readable status. Summarize security posture across all students in a single report. Provide small, standalone cyber security utilities that reinforce good security habits.

The project is intentionally built without external frameworks or a database, relying only on Python's standard library and a flat text file (students.txt) for storage — making it easy to read, run, and extend for beginners.

Features

Student Record Management

- Add Student — capture name, ID, branch, and email with input validation (unique numeric ID, valid email format, non-empty fields).
- View Students — display all saved records in a clean, formatted list.
- Search Student — look up a record by name (partial match) or exact student ID.
- Delete Student — remove a record with an explicit yes/no confirmation step.

Security Assessment

Asks four questions per student: MFA enabled, password length, system updated, antivirus installed. Calculates a weighted Security Score out of 100. Maps the score to a category: Excellent, Good, Moderate, or Poor. Saves the result back into that student's record.

Reporting

Generate Report — shows total students, number assessed, average security score, individual scores, and a list of students flagged with a Poor rating.

Cyber Security Tools (3 standalone features)

- Password Strength Checker — checks length, uppercase/lowercase, digits, and special characters, then rates the password from Very Weak to Very Strong.
- Username Generator — generates five secure, randomized username suggestions from a given name. Security
- Awareness Quiz — a 5-question multiple-choice quiz on phishing, MFA, password reuse, and safe browsing, with a scored awareness level at the end.

File Storage

All records persist in students.txt using a simple pipe-delimited (|) format. Full create, read, update, and delete operations are supported on the file.

User Interface

Fully menu-driven, numbered options, returns to the main menu after every action. Consistent formatted headers and dividers for readability. Input validation and error handling throughout — invalid menu choices, non-numeric IDs, duplicate IDs, malformed emails, and bad yes/no input are all caught and re-prompted instead of crashing the program.

Technologies Used

Language: Python 3 Standard Library

Modules: os — checking for and creating students.txt re — email format validation and password character checks random and string — generating username suggestions

Storage: Plain text file (students.txt), no database or external packages required Environment:

Runs in any terminal with Python 3 installed (no third-party dependencies)

Program Flow

1. Startup — the program checks whether students.txt exists and creates it if not.
2. Main Menu — the user is shown eight options: Add Student, View Students, Search Student, Delete Student, Security Assessment, Generate Report, Cyber Security Tools, and Exit.
3. Add Student — collects and validates student details, then appends a new record to the file.
4. View / Search / Delete — all three load the current records from students.txt into memory, operate on them (display, filter, or remove), and — for deletions — rewrite the file with the updated list.

5. Security Assessment — looks up a student by ID, asks the four security questions, computes the score and status, then updates that student's stored record.
6. Generate Report — reads all records, computes aggregate statistics (totals, average score, poor-rated students), and displays them on screen.
7. Cyber Security Tools — opens a sub-menu for the three independent utilities (password checker, username generator, quiz), each operating without needing existing student data.
8. Exit — closes the program safely; all changes up to that point are already saved on disk.

The loop repeats after every action, returning to the main menu until the user selects Exit.

Challenges Faced

Designing a reliable text-based storage format: Since there's no database, choosing a delimiter (|) that wouldn't conflict with real data, and handling malformed or incomplete lines gracefully, took some care. Input validation without external libraries: Writing regex patterns for email validation and password strength checks, and building reusable input-loop functions so invalid input never crashes the program. Keeping the score formula simple yet meaningful: Balancing the four security factors (MFA, password length, updates, antivirus) into a 100-point scale that still produces intuitive, well-distributed results across the four status categories. Maintaining state across menu navigation: Since the program reloads data from the file on each operation rather than keeping everything in memory, every module (especially delete and update) had to be carefully synchronized with what's actually saved on disk. Designing for a non-technical user: Making error messages clear and consistent, and ensuring the program always returns control to the main menu instead of exiting unexpectedly.

Learning Outcomes

Practiced structuring a multi-module Python application using clear function separation rather than one large script. Gained experience with file handling in Python — reading, appending, and rewriting structured text data. Strengthened understanding of input validation and error handling patterns that keep a console application stable. Applied basic cyber security concepts practically: what makes a password strong, why MFA matters, and how to communicate security posture through a simple scoring system. Learned to design a menu-driven CLI that is intuitive and forgiving for non-technical users. Understood the trade-offs of using flat files for storage versus a database, and when each is appropriate for small-scale projects.